

Module 2: Transactions and Instructions

Building and Submitting Transactions

Transactions vs Instructions

Solana Transaction Model

- **Transaction:** Container for multiple operations
- **Instructions:** Individual operations within a transaction
- **Key Difference:** One transaction can execute multiple instructions atomically

EVM Comparison

- **EVM:** Each transaction calls one contract function
- **SVM:** Transactions can batch multiple operations

Transaction Structure

Core Components

- **Signatures:** Array of signer public keys
- **Message:** Contains all transaction data
 - Recent blockhash
 - Account addresses
 - Instructions array

Instruction Format

- **Program ID:** Which program to execute
- **Accounts:** Which accounts the instruction uses
- **Data:** Instruction-specific parameters

Transaction Lifecycle

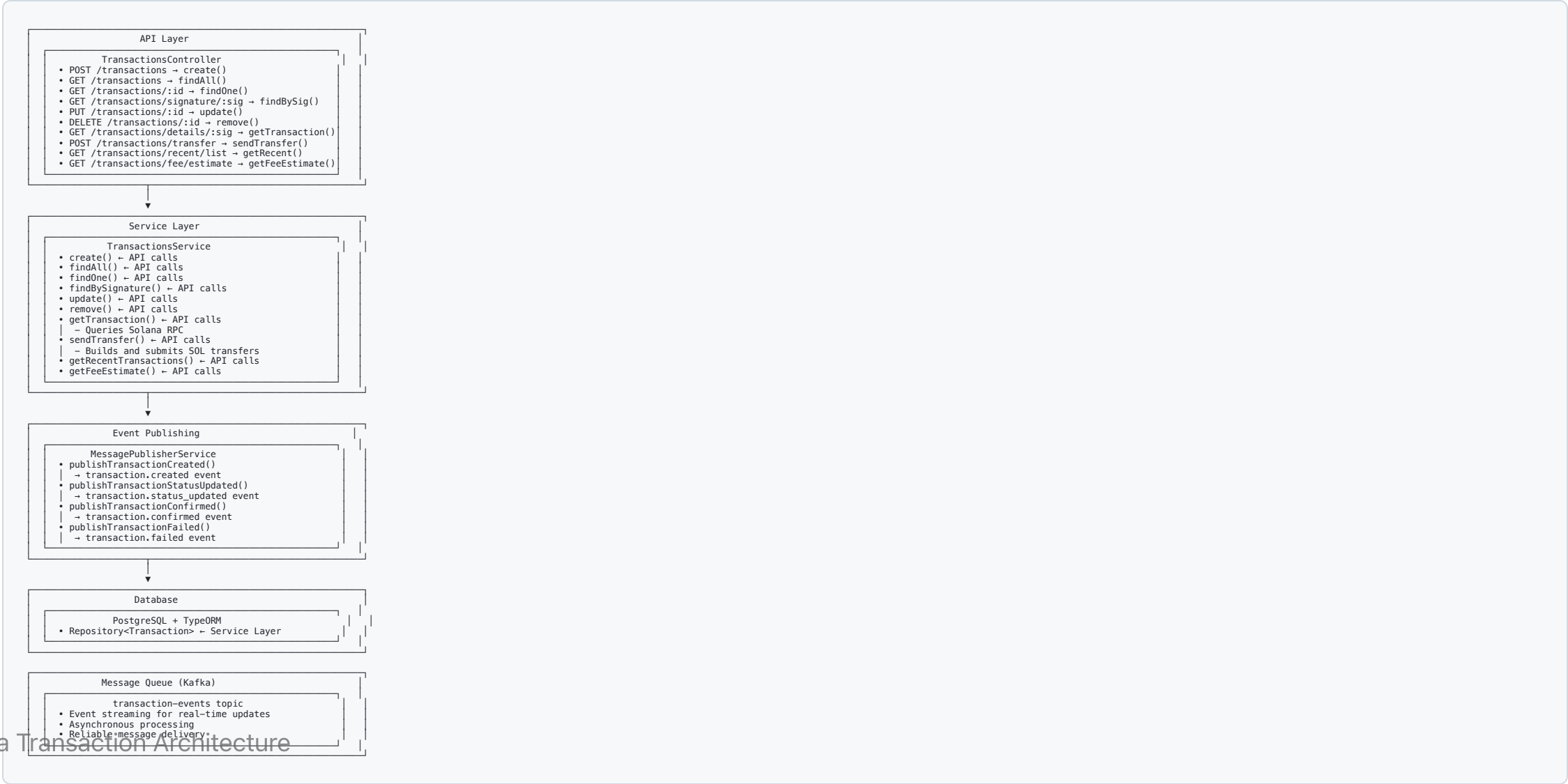
Status Flow

1. **PENDING**: Transaction created, not yet submitted
2. **CONFIRMED**: Successfully executed on-chain
3. **FAILED**: Execution failed with error

Key Events

- Transaction created
- Status updates
- Confirmation notifications
- Failure alerts

Architecture Overview



Transaction with Multiple Instructions

```
// Create multiple instructions
const instructions = [
  SystemProgram.transfer({...}),
  TokenProgram.transfer({...}),
  // ... more instructions
];

// Add to transaction
const transaction = new Transaction();
transaction.add(...instructions);

// Send transaction
const signature = await sendAndConfirmTransaction(connection, transaction, signers);
```

Error Handling

Common Transaction Errors

- **Insufficient Funds:** Not enough SOL for transfer + fees
- **Invalid Account:** Account doesn't exist or wrong owner
- **Program Error:** Instruction execution failed
- **Timeout:** Transaction not confirmed in time

Retry Mechanisms

- Exponential backoff
- Fee adjustment
- Alternative RPC endpoints
- Dead letter queue for persistent failures

Performance Considerations

Optimization Strategies

- Batch multiple operations in single transaction
- Use efficient instruction ordering
- Implement proper fee management
- Cache frequently used data

Monitoring

- Transaction success rates
- Confirmation times
- Fee efficiency
- Error patterns

Security Best Practices

Transaction Security

- Validate all input parameters
- Verify account ownership
- Implement proper authorization
- Use secure key management

Replay Protection

- Recent blockhash prevents replay
- Unique transaction signatures
- Proper nonce handling

Next Steps

Module 2 Implementation Tasks

-  Basic transaction CRUD
-  SOL transfer functionality
-  Multi-instruction transactions
-  Advanced fee management
-  Token transfer transactions

Related Modules

- **Module 1:** Accounts & Programs
- **Module 3:** Token Standards
- **Module 5:** Fee Mechanism

Thank You!

Questions?

Ready to explore Token Standards?

Next: Module 3 - Token Standards 